



SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

Module 50.002 | Team 12 | Project : **RhythMania**

**Instructions:*

1. **Directly** work on your report in this Google doc. Do not work elsewhere and paste it here.
2. Use the template formatting, if it is a heading (e.g., Heading 1, Heading 2 etc.), format it as such so the **Table of Contents** will reflect it.
3. The point of this Google doc is to be a **live** document that you continually update as you work on your 1D project. The teaching staff will be taking a look at these documents as you work on them to see how your team is progressing.

Name	Student ID
Rahmthulla Salman Munthaseer	1006564
Tan Yuan Xiang Elroy	1006569
Lee Wei Xian	1008773
Ye Jianrong, Jayden	1008899
Jarrood Low Chan Sian	1008976
Ng Wen Yin	1009135
Wong Kysin Natalie	1007309

Table of Contents

[Table of Contents](#)

[Introduction](#)

[Game Design](#)

[Description of the Game](#)

[User Manual](#)

[Design Inspirations](#)

[Electronic Design](#)

[Input/Output](#)

[Datapath](#)

[FSM](#)

[Budget](#)

[Sustainability](#)

[Summary](#)

[References](#)

[Appendix](#)

[1. ALU Design and Tests](#)

[2. Prototype code + Repo link](#)

[3. Project Management Log: Team Tasks](#)

[4. Components' Specifications](#)

Introduction



This project is an interactive rhythm game inspired by Rhythm Heaven designed to evaluate a player's musical timing, reaction speed and beat synchronisation. The system is housed in a laser-cut wooden enclosure that integrates input, audio and visual feedback components into a compact physical interface.

Gameplay begins when the player presses the button in the middle to initiate the game. A set of 4 LEDs emits a metronome signal at a defined 60 BPM (beats per minute). The player must press the button in precise alignment with each beat from the displayed rhythm. The FPGA continuously measures the difference between the expected beat interval and the player's input, enabling real time accuracy evaluation.

Visual feedback is provided through the two 7 segment displays and the LED arrays around it. The 7 segment display tracks and updates the player's score dynamically based on timing precision.

Combining the numerical scoring system with the visual cues by the LEDs creates a feedback loop that allows players to calibrate their timing instinctively.

Game Design

Description of the Game

RhythmMania is a rhythm-based game that challenges players' sense of timing and auditory memory. The game uses a buzzer to play a sequence of rhythmic patterns, which the player must carefully listen to and replicate using a single button.

Players earn points based on how accurately they reproduce the rhythm, including both timing and sequence precision. As the game progresses, the rhythms can become more complex, increasing the level of difficulty and testing the player's reaction speed and concentration.

User Manual

RhythmMania is designed to be simple and intuitive for users of all ages. Follow the steps below to operate the prototype:

1. Starting the Game

- Press the main button once to start.

2. First Countdown Phase

- The 4 countdown LEDs will light up in sequence.
- This signals that the rhythm playback is about to begin.
- Prepare to listen carefully.

3. Call Phase (System Plays Rhythm)

- The FPGA plays a 1-bar (4-count) rhythm.
- This is communicated through:
 - Flashing LEDs
 - Audio feedback (buzzer)
- The countdown LEDs continue flashing as a metronome to indicate timing.

4. Second Count Down Phase

- The 4 countdown LEDs continue to flash.
- The player should be ready to respond after the 4 beat countdown.

5. Response Phase (Player inputs)

- After the countdown finishes, the player must replicate the rhythm using the big button.
- Press the button in the same timing and rhythm as the buzzer sound heard in the earlier call phase.
- Try to match both the speed and spacing of each beat.
- If the rhythm played by the user is good, the green LED will light up. If it is okay, the yellow LED will light up. If they miss the note, the red LED will light up.
- The game will return to the Call Phase (3) until all 5 rounds have been played.

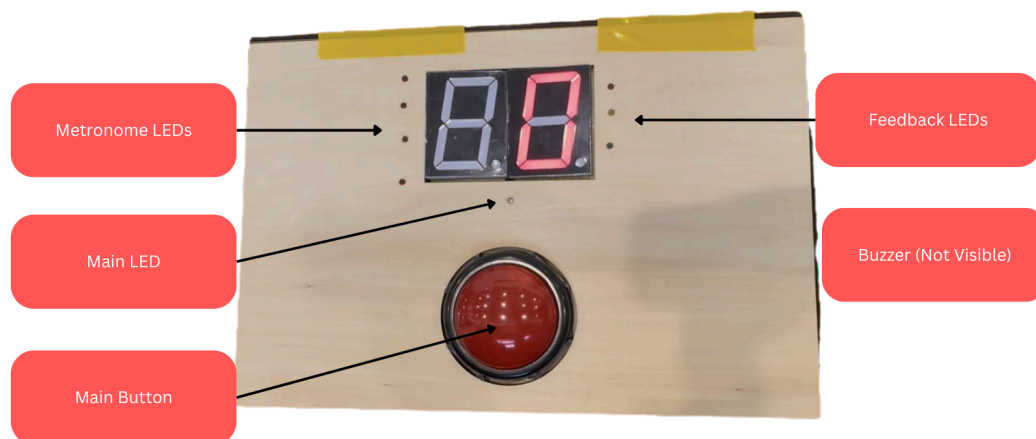
6. Scoring System

- The scoring system is updated after every note the user plays.
- Once all 5 rounds on rhythm calling and response phases have completed, the final score is displayed.
- Press the button once after the last game has ended to reset.

7. Restarting the Game

- Press the button once again to start the first countdown phase.

UI & UX



Labeled diagram of our inputs and outputs

1. User Inputs

The game is designed with a minimal and intuitive input system to ensure ease of use and focus on rhythm interaction:

- **1 × Big Dome Push Button (100mm)**
 - Serves as the primary gameplay input
 - Used by the player to replicate the rhythm during the [Response Phase](#)
 - Large size enhances accessibility and tactile feedback, improving user experience

2. System Outputs

The system provides multimodal feedback (visual and auditory, and haptic) to enhance user engagement and accuracy:

- **1 × LED (Main Rhythm Indicator)**
 - Displays the rhythm pattern during the [Call Phase](#)
- **1 × Passive Buzzer**
 - Produces audio cues for rhythm playback
 - Reinforces timing through sound
- **2 × 1-Digit 7-Segment Displays**
 - Displays the player's current score
 - Allows real-time performance tracking
- **3 × Feedback LEDs**
 - **Green** → Good timing
 - **Yellow** → Ok timing
 - **Red** → Miss timing
 - Provides immediate performance feedback per beat
- **4 × Countdown / Metronome LEDs**
 - Guides the player through timing phases
 - Acts as a visual metronome during gameplay
 - Signals transitions between phases

3. Enclosure materials

Wood

- Used to construct the outer casing of the prototype
- Provides structural stability and arcade-like aesthetic

4. Photo of prototype



Design Inspirations



YouTube Rhythm Hell

YouTube The Official Square Game | Name Each Square on Beat (Medium)

Inspired by similar videos of rhythm games, we wanted to make our own rhythm game to test people's sense of rhythm.

We wanted to modify it to be more user friendly with the limited output LEDs we had, as well as adding a challenge of memory. As such, we added another 4 beats to count them in before they start playing, so that players need to be more patient and remember the rhythm.

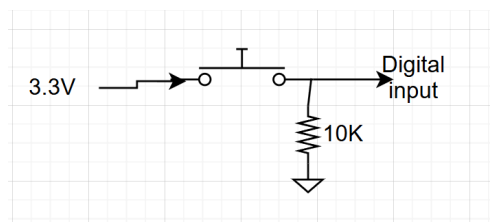
Electronic Design

Lots of diagrams are expected here.

Input/Output

- Inputs
 - **1x Big Dome Push Button (100mm):** The primary gameplay input for repeating the rhythm.

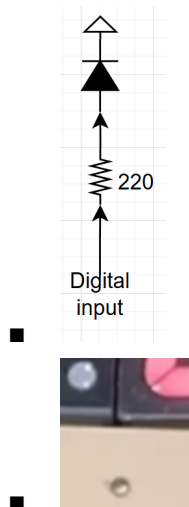
- 1 - When the output of the button is pressed. This is fed to a button conditioner and edge detector
 - 0 - Otherwise



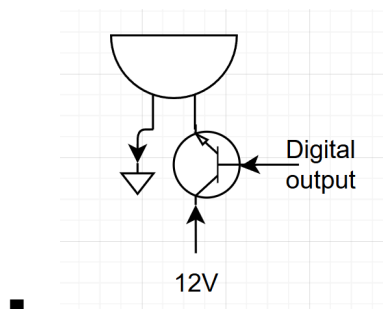
- To ensure there is no power on surge there is a Physical Pull-Down Resistor added to the circuit



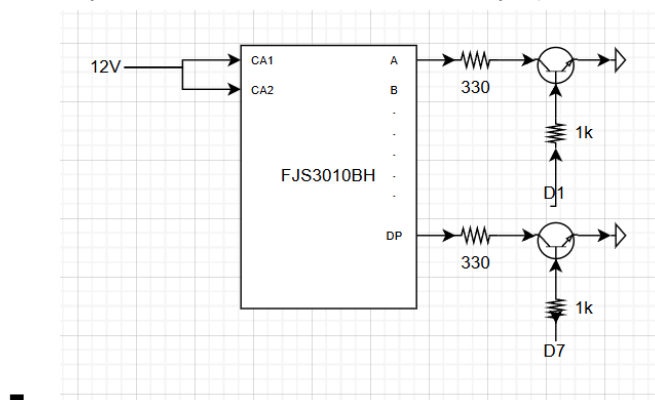
- Outputs
 - **1x white LED:** For the main rhythm visual.
 - 1 - When the global timer equals to the note timestamp, with an extended pulse
 - 0 - Otherwise



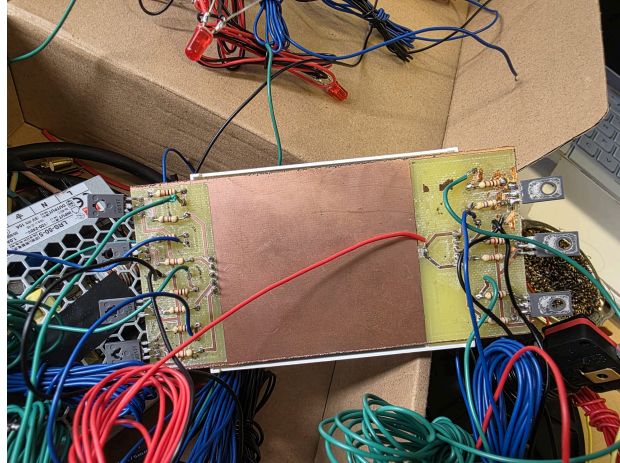
- **1x 12V Passive Buzzer:** For audio reinforcement of the note display rhythm.
 - 1 - When the global timer equals to the note timestamp, with an extended pulse
 - 0 - Otherwise



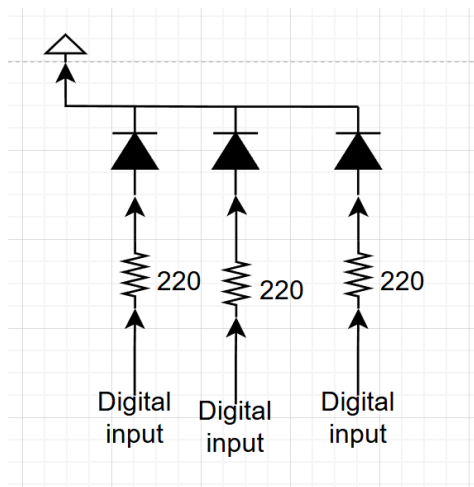
- **2x 1-Digit 7-Segment Display:** To show the current score.
 - 1/0 - Always displays the value of Score Reg, output per segment depends on the number being displayed



- To reduce electronic placement issue a PCB was created

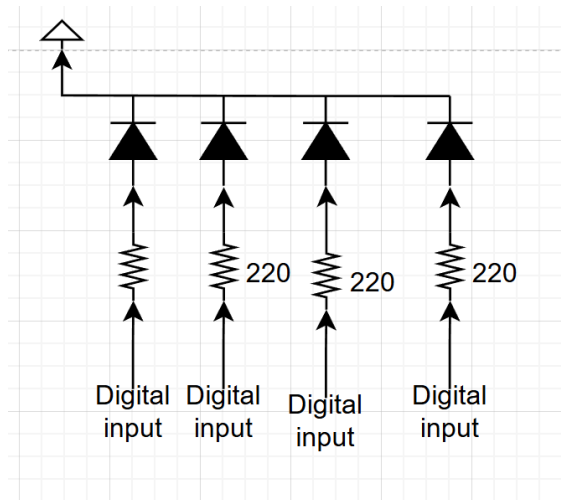


- **3x LED:** Accuracy feedback (Green = Good, Yellow = OK, Red = Miss).
 - 1 (Yellow and Green LEDs) - When a button hit is registered within the respective Ok/Good button hit windows, with pulse extension
 - 1 (Red LED) - When the Global timer is 100ms past the hit window of the note
 - 0 - Otherwise

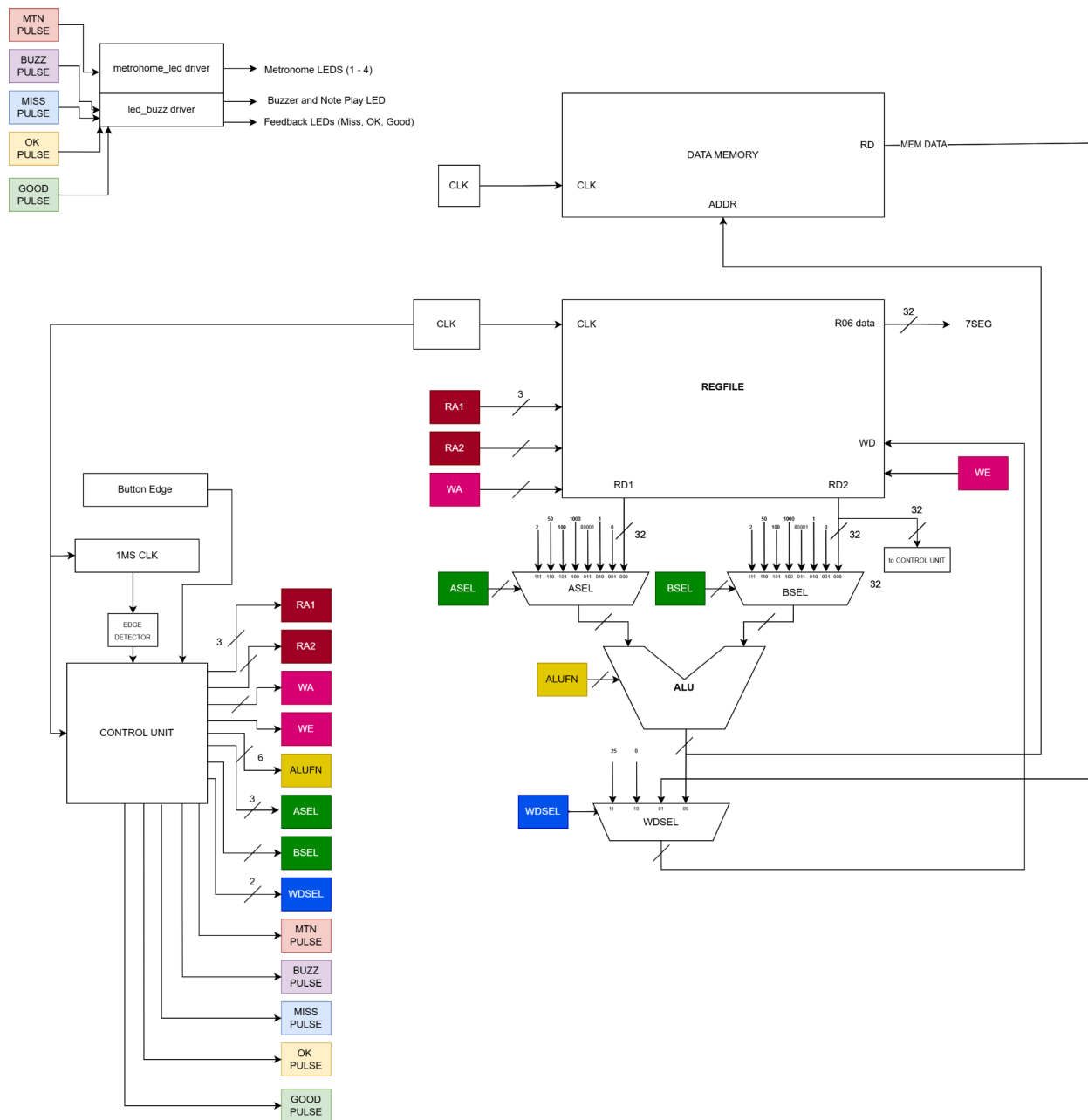




-
- **4x Red LED Metronome LEDs:** To signal the speed on the rhythm
 - 1 - whenever each LED's beat is active (On for 1 sec each time)
 - 0 - Otherwise



Datapath

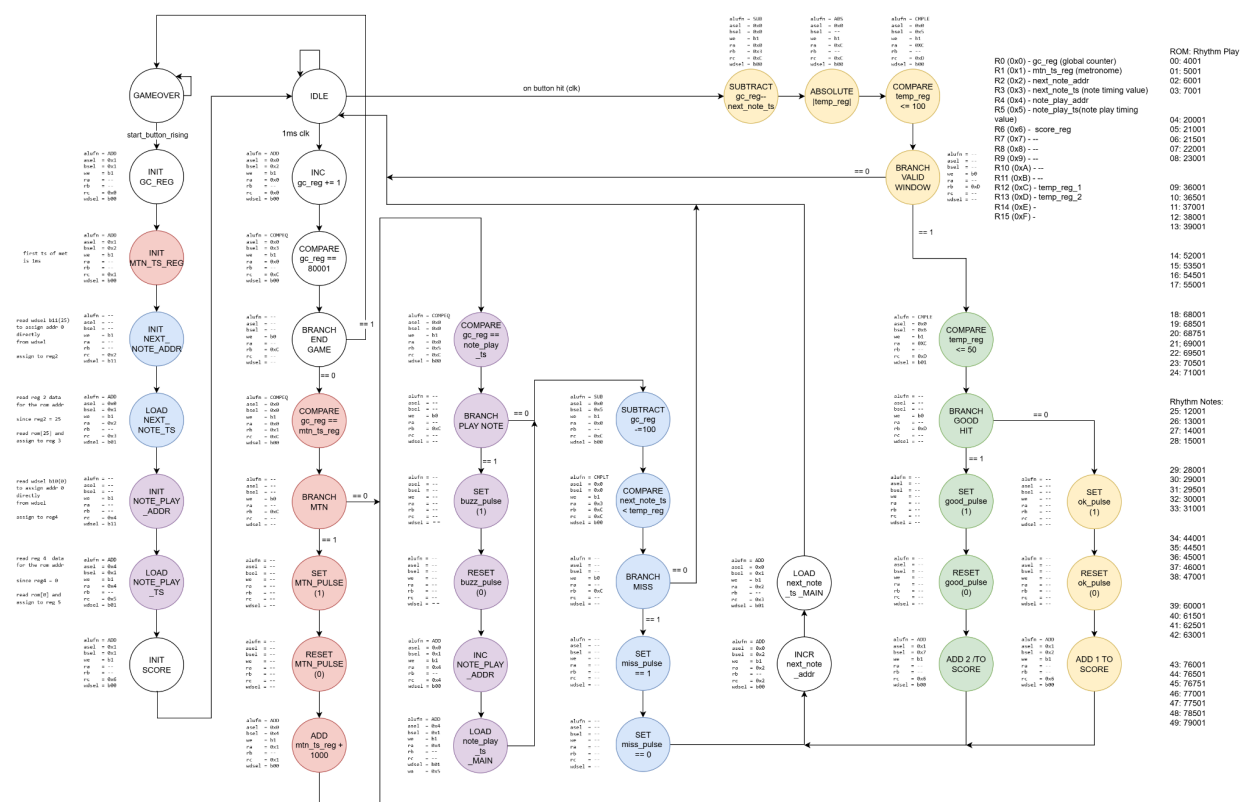


- Control Unit (FSM)
 - The main brains of the machine. Decides what the machine needs to do through outputting the correct signals
 - RA1 & RA2 - Specifies which address RD1 and RD2 reads from
 - WA - Specifies which address to write to
 - WE - Enables writing to registers
 - ALUFN - Which function's output the ALU decides to output

- ASEL & BSEL - Specifies if either register data or constants are read
 - WDSEL - Specifies if either ALU outputs or constants are read
 - MTN_PULSE - Signal for metronome_led driver to pulse metronome LEDs
 - BUZZ_PULSE - Signal for led_buzz driver to pulse buzzer and note play LEDs
 - MISS_PULSE - Signal for led_buzz driver to pulse MISS LEDs
 - OK_PULSE - Signal for led_buzz driver to pulse OK LEDs
 - GOOD_PULSE - Signal for led_buzz driver to pulse GOOD LEDs
- Clk
 - The main 10MHz clock (Generated from 100MHz clk to meet timing constraints)
- 1ms Clk
 - Outputs a rising edge every 1ms for incrementing gc_reg
- Button Edge
 - Outputs 1 when a rising edge of a button press is detected
- Regfile
 - Stores the register data to be regularly used
 - R0 (0x0) - gc_reg (global counter)
 - R1 (0x1) - mtn_ts_reg (metronome)
 - R2 (0x2) - next_note_addr (address for ROM)
 - R3 (0x3) - next_note_ts (note timing value for actual button hit detection)
 - R4 (0x4) - note_play_addr (address for ROM)
 - R5 (0x5) - note_play_ts (note play timing value for displaying rhythm)
 - R6 (0x6) - score_reg (to be constantly outputted to 7 seg)
 - R7 (0x7) - --
 - R8 (0x8) - --
 - R9 (0x9) - --
 - R10 (0xA) - --
 - R11 (0xB) - --
 - R12 (0xC) - temp_reg_1 (Default temp reg for most use cases, like storing comparison boolean results)
 - R13 (0xD) - temp_reg_2 (Used when R12 is storing the absolute difference for sequential comparisons)
 - R14 (0xE) - Default register value (if register was erroneously not assigned)
 - R15 (0xF) - --
- Data Memory (ROM)
 - Stores the data of both note play timestamps, as well as next note timestamps
 - RD - The data from the ROM that's outputted based on the address from the ALU out
- ASEL & BSEL
 - Chooses either the regfile data or constants to input to the ALU
 - 000 (RD1/RD2) - Read the register data

- 001 (0) - General constant
 - 010 (1) - General constant
 - 011 (80001) - The ending ms timestamp to check for the end of the game
 - 100 (1000) - Constant to increment mtn_reg
 - 101 (100) - Constant to add to gc_reg to check within OK timestamp
 - 110 (50) - Constant to add to gc_reg to check within Good timestamp
 - 111 (2) - Constant to add to score_reg for Good button hit
- ALU
 - In charge of the math logic
 - 000000 - ADD
 - 000001 - SUBTRACT
 - 000010 - MULTIPLY
 - 011000 - AND
 - 011110 - OR
 - 010110 - XOR
 - 011010 - A (pass)
 - 100000 - SHL
 - 100001 - SHR
 - 100011 - SRA
 - 110011 - CMPEQ
 - 110101 - CMPLT
 - 110111 - CMPLE
 - 111111 - ABSOLUTE (Custom ALU fn)
- WDSEL
 - Chooses between ALU out, ROM data out, or a constant to write to register, if write enabled
 - 00 - ALU output
 - 01 - ROM output
 - 10 - 0 (for initialising note_play_addr)
 - 11 - 25 (for initialising next_note_addr)

FSM



Default Values:

asel = b0 (Read from Regfile RA1)
 bsel = b0 (Read from Regfile RA2)
 wdsel = b0 (Write ALU output)
 we = b0 (Write Enable OFF - protects registers)
 wa = hE (Write Address 14 - safe dump register)
 mtn_pulse = 0 (Metronome Pulse OFF)
 buzz_pulse = 0 (Buzzer and Note Play Pulse OFF)
 miss_pulse = 0 (Miss LED Pulse OFF)
 ok_pulse = 0 (OK LED Pulse OFF)
 good_pulse = 0 (Good LED Pulse OFF)

Red: Metronome logic

Blue: Miss logic

Purple: Note display logic

Yellow: Ok hits logic

Green: Good hits logic

White: Misc logic (ROM, score)

Default States	Control Signals	Description
GAMEOVER	(Default Signals)	The default state when switching on the FPGA. Waits for a button press to start the game.
IDLE	(Default Signals)	Default state when the game is in progress. Waits for either a button press or a 1ms clk tick.
Initialization States	Control Signals	Description
INIT_GC_REG	alufn = b000000 asel = h1 bsel = h1 we = 1 wa = h0 wdsel = b00	Initialises gc_reg to 0.
INIT_MTN_TS_REG	alufn = b000000 asel = h1 bsel = h2 we = 1 wa = h1	Initialises mtn_ts_reg to 1.

	wdsel = b00	
INIT_NEXT_NOTE_ADDR	we = 1 wa = h2 wdsel = b11 (Const 25)	Initialises next_note_addr to 25.
LOAD_NEXT_NOTE_TS_INIT	alufn = b000000 asel = h0 bsel = h1 ra1 = h2 we = 1 wa = h3 wdsel = b01 (ROM)	Loads the first next_note_ts from ROM.
INIT_NOTE_PLAY_ADDR	we = 1 wa = h4 wdsel = b10 (Const 0)	Initialises note_play_addr to 0.
LOAD_NOTE_PLAY_TS_INIT	alufn = b000000 asel = h0 bsel = h1 ra1 = h4 we = 1 wa = h5 wdsel = b01 (ROM)	Loads the first note_play_ts from ROM.

INIT_SCORE	alufn = b000000 asel = h1 bsel = h1 we = 1 wa = h6 wdsel = b00	Initialises Score to 0.
INC_GC_REG	alufn = b000000 asel = h0 bsel = h2 ra1 = h0 we = 1 wa = h0 wdsel = b00	Increases gc_reg by 1ms.
End Game Check States	Control Signals	Description
COMPARE_GC_END_TS	alufn = b110011 asel = h0 bsel = h3 ra1 = h0 we = 1 wa = hC wdsel = b00	Checks if it's the end timestamp of 80001 and stores the result to temp_reg_1.
BRANCH_END_GAME	ra2 = hC	Reads temp_reg_1. If it's the end game timestamp, branch to GAMEOVER. Else continue.
Metronome States	Control Signals	Description

COMPARE_GC_MTN	alufn = b110011 asel = h0 bsel = h0 ra1 = h0 ra2 = h1 we = 1 wa = hC wdsel = b00	Checks if gc_reg is currently a metronome timestamp and stores the result to temp_reg_1.
BRANCH_MTN	ra2 = hC	Reads temp_reg_1. If it's the metronome timestamp, branch to SET_MTN_PULSE_1. Else continue to COMPARE_GC_NOTE_PLAY_TS.
SET_MTN_PULSE_1	mtn_pulse = 1	Sets mtn_pulse to 1 for metronome_led module to read.
RESET_MTN_PULSE_0	mtn_pulse = 0	Resets the mtn_pulse to 0.
ADD_MTN_TS_1000	alufn = b000000 asel = h0 bsel = h4 ra1 = h1 we = 1 wa = h1 wdsel = b00	Increases mtn_ts_reg by 1000ms.
Note Display States	Control Signals	Description

COMPARE_GC_NOTE_PLAY_TS	alufn = b110011 asel = h0 bsel = h0 ra1 = h0 ra2 = h5 we = 1 wa = hC wdsel = b00	Checks if gc_reg is currently a note play timestamp and stores the result to temp_reg_1.
BRANCH_PLAY_NOTE	ra2 = hC	Reads temp_reg_1. If it's the note play timestamp, branch to SET_BUZZ_PULSE_1. Else continue to SUBTRACT_GC_100.
SET_BUZZ_PULSE_1	buzz_pulse = 1	Sets buzz_pulse to 1 for led_buzz module to read.
RESET_BUZZ_PULSE_0	buzz_pulse = 0	Resets the buzz_pulse to 0.
INC_NOTE_PLAY_ADDR	alufn = b000000 asel = h0 bsel = h2 ra1 = h4 we = 1 wa = h4 wdsel = b00	Increases the note_play_addr by 1.

	alufn = b000000 asel = h0 bsel = h1 ra1 = h4 we = 1 wa = h5 wdsel = b01 (ROM)	Loads the next note play timestamp into note_play_ts.
LOAD_NOTE_PLAY_TS_MAIN		
Miss Check States	Control Signals	Description
	alufn = b000001 asel = h0 bsel = h5 ra1 = h0 we = 1 wa = hC wdsel = b00	Subtracts gc_reg by 100ms and stores it into temp_reg_1.
SUBTRACT_GC_100		
	alufn = b110101 asel = h0 bsel = h0 ra1 = h3 ra2 = hC we = 1 wa = hC wdsel = b00	Reads temp_reg_1. Checks if gc_reg is 100ms past the next_note_ts (a miss condition) and stores the result to temp_reg_1.
COMPARE_NEXT_NOTE_TS_TEMP		
BRANCH_MISS	ra2 = hC	Reads temp_reg_1. If it's past the note play timestamp, branch to SET_MISS_PULSE_1. Else continue to IDLE.

SET_MISS_PULSE_1	miss_pulse = 1	Sets miss_pulse to 1 for led_buzz module to read.
SET_MISS_PULSE_0	miss_pulse = 0	Resets the miss_pulse to 0.
Button Press States	Control Signals	Description
SUBTRACT_GC_NEXT_NOTE_T S	alufn = b000001 asel = h0 bsel = h0 ra1 = h0 ra2 = h3 we = 1 wa = hC wdsel = b00	Finds the difference between the button hit timestamp and the next note timestamp. Stores to temp_reg_1.
ABSOLUTE_TEMP	alufn = b111111 asel = h0 ra1 = hC we = 1 wa = hC wdsel = b00	Reads temp_reg_1 Finds the absolute difference and stores it to temp_reg_1.
COMPARE_TEMP_100	alufn = b110111 asel = h0 bsel = h5 ra1 = hC we = 1 wa = hD wdsel = b00	Reads temp_reg_1. Checks if temp_reg_1 (absolute time diff) is within 100ms (OK hit condition). Stores the result to temp_reg_2.
BRANCH_VALID_WINDOW	ra2 = hD	Reads temp_reg_2. If within an OK window, branch to COMPARE_TEMP_50.

		Else continue to IDLE.
COMPARE_TEMP_50	alufn = b110111 asel = h0 bsel = h6 ra1 = hC we = 1 wa = hD wdsel = b00	Checks if temp_reg_1 (abs time diff) is within 50ms (Good hit condition). Stores the result to temp_reg_2.
BRANCH_GOOD_HIT	ra2 = hD	Reads temp_reg_2. If within Good window, branch to SET_GOOD_PULSE_1. Else branch to SET_OK_PULSE_1.
Good Hit States	Control Signals	Description
SET_GOOD_PULSE_1	good_pulse = 1	Sets good_pulse to 1 for led_buzz module to read.
RESET_GOOD_PULSE_0	good_pulse = 0	Resets good_pulse to 0.
ADD_2_SCORE	alufn = b000000 asel = h0 bsel = h7 ra1 = h6 we = 1 wa = h6 wdsel = b00	Adds 2 to the score.
Ok Hit States	Control Signals	Description
SET_OK_PULSE_1	ok_pulse = 1	Sets ok_pulse to 1 for led_buzz module to read.
RESET_OK_PULSE_0	ok_pulse = 0	Resets ok_pulse to 0.

	alufn = b000000 asel = h0 bsel = h2 ra1 = h6 we = 1 wa = h6 wdsel = b00	
ADD_1_SCORE		Adds 1 to the score.
Load Next Notes States	Control Signals	Description
	alufn = b000000 asel = h0 bsel = h2 ra1 = h2 we = 1 wa = h2 wdsel = b00	
INCR_NEXT_NOTE_ADDR		Increments the next_note_addr by 1.
	alufn = b000000 asel = h0 bsel = h1 ra1 = h2 we = 1 wa = h3 wdsel = b01 (ROM)	
LOAD_NEXT_NOTE_TS_MAIN		Loads the next note timestamp into next_note_ts.

Budget

1x Main Big Red Button - \$12.80
3x 7 Segment 1 Digit Red Common Anode - 3 @ \$8 ea = \$24
1x Buzzer 6-24V High Pitch Beep Beep - \$2.50
1x 12V buzzer - \$1.80
1x 5V buzzer - \$1.50
Total - \$42.30

Streamlining Efficiency and Cost

Our group has implemented instances of simplified circuit design. One such example is that we programmed the ALU in such a way that all sub units feed to a Single Output MUX.

```
case(alufn[5:0]) {  
    6b000000: alu_result = adder_out  
    6b000001: alu_result = adder_out    // SUB shares adder output  
    6b000010: alu_result = multiplier_out  
    6b011000: alu_result = boolean_out  
    ...  
    6b111111: alu_result = absolute_out  
}  
out = alu_result
```

The sub units (adder, boolean, compare, shifter, multiplier, absolute) are all computed simultaneously every cycle. The “case” statement acts as a single output multiplexer that selects the relevant result rather than enabling/disabling the individual subunits that are based on the operation. This simplifies the control logic to 1 selector signal only (ALUFN) feeding one MUX.

As well as that, we directly used bitwise reductions, rather than comparison operators, in codes even outside the ALU. As multi-bit equality is expensive hardware wise, it increases complexity of the hardware. As such, direct bitwise reductions like (*if &counter.q*) to check for a DFF that is full reduces complexity and increases hardware efficiency.

We also used a 1ms clk to reduce the size of timestamps so that logic can be calculated more quickly with bits sizes that are within the 32 bit limit, while still maintaining enough granularity in calculations to still be meaningfully accurate for a rhythm game.

Hardware wise, we utilised wood planks that were sourced from extra parts from our previous projects, meaning no new raw materials were sourced just for the casing, thus reducing costs of the project.

Enhancing User Engagement

Our game system is designed with simplicity and accessibility in mind. We used a large, ergonomically designed button with little force required for input so that users can respond quickly and comfortably, minimising input delay that could potentially give the player a lower score.

We also incorporated two large 7 segment displays that can provide a clear view of the player's score, with the multicoloured LED array on the left side on the panel to deliver feedback on the player's accuracy when pressing the button. Another set of red LED's are present to help visualise the metronome that is inside the box.

The gameplay itself is also simple, just press the button on time to gain points. This minimal instruction set enhances user experience as a result.

Inside the box itself, all wiring is carefully managed and secured within the box itself to maintain a clean and organised exterior.

Sustainability and Inclusivity

This project integrates sustainability through three key pillars: Responsible consumption and production (SDG 12), Quality Education (SDG 4) and Reduced Inequalities (SDG 10).

Firstly, as mentioned in streamlining costs, our game utilises wood planks that were sourced from extra parts from our previous projects, meaning no new raw materials were sourced just for the casing. We also designed it in such a way that after the showcase, we are able to disassemble the parts relatively easily and leave the recyclable parts of our project for future use in other projects, which satisfies SDG 12.

Next, our game functions as an educational tool for the players. It allows them to train their rhythm perception, memory and reaction speed, which are skills that can help in cognitive development. The structured gameplay also mirrors established pedagogical techniques used in music teaching by allowing the player to listen, remember and replicate, which satisfies SDG 4.

Finally, the design of our game system also promotes greater inclusivity. We incorporated items like a large button for users with limited fine motor control to still enjoy our game, and the large 7 segment displays allows those who are visually challenged to see their score better. The buzzer in our game acts as an auditory indicator, but we also added a set of red LED's that flashes in accordance with the buzzer to help those who are hard of hearing to play without any handicap, thus satisfying SDG 10.

Summary

This 2D project successfully translates our functional 1D prototype into a highly optimized, user-centric, and sustainable rhythm game. Hardware efficiency was maximized across multiple domains: dynamic power consumption was drastically reduced by scaling the FPGA's system clock down to 10MHz via a custom MMCM, while gate-level logic was streamlined using single-multiplexer ALU routing and bitwise reduction operators to minimize look up table utilization. User engagement (UI/UX) was elevated through an accessible, zero-latency gameplay loop, supported by a highly tactile arcade interface and instantaneous, multi-colored LED performance feedback. Furthermore, the prototype champions sustainability and inclusivity (aligning with UN SDGs 4, 10, and 12) through the utilization of upcycled, modular casing materials, and a multimodal (auditory and visual) feedback system that ensures the experience remains engaging for users with varying visual, auditory, or motor abilities.

References

<https://natalieagus.github.io/50002/>

<https://alchitry.com/tutorials/lucid-reference/>

<https://www.youtube.com/watch?v=a-El30uucng> Rhythm Hell

Appendix

1. ALU Design and Tests

Overall ALUFN signals	Function Implemented In
000000 - ADD	adder.luc, adder_rca.luc
000001 - SUBTRACT	adder.luc, adder_rca.luc
000010 - MULTIPLY	multiplier.luc
011000 - AND	boolean.luc
011110 - OR	boolean.luc
010110 - XOR	boolean.luc
011010 - A (pass)	boolean.luc
100000 - SHL	shifter_left.luc, shifter.luc
100001 - SHR	shifter_left.luc, shifter.luc
100011 - SRA	shifter_left.luc, shifter.luc
110011 - CMPEQ	compare.luc
110101 - CMPLT	compare.luc
110111 - CMPLE	compare.luc
111111 - ABSOLUTE (custom func)	absolute.luc

In this game, only ADD, SUBTRACT, CMPEQ, CMPLT, CMPLE and ABSOLUTE modules were used.

Design:

Takes in 32 bit **signed** inputs *a* and *b* and 6 bit input *alufn*, and outputs 32 bit **signed** output *out*

- Due to use of the absolute function (which returns the positive of the input value), the whole ALU is required to take in and give out signed signals (for consistency in arithmetic calculations). Although this means the max positive value is only $2^{31} - 1$, it is not a problem for RhythmMania's scope
- Specific implementation of submodules listed below

Tests:

- ALU submodules were tested individually via testbench
- Overall ALU for this game was tested and monitored via an early stage datapath testbench
- Datapath testbench was fed button_input to start game and team monitored fsm states, register contents alongside ALU operations across many clks
- Testbench allowed us to identify erroneous alufn signals and unexpected ALU output due to initial unsigned input and output

ALU Sub-Module Implementation Design and Tests

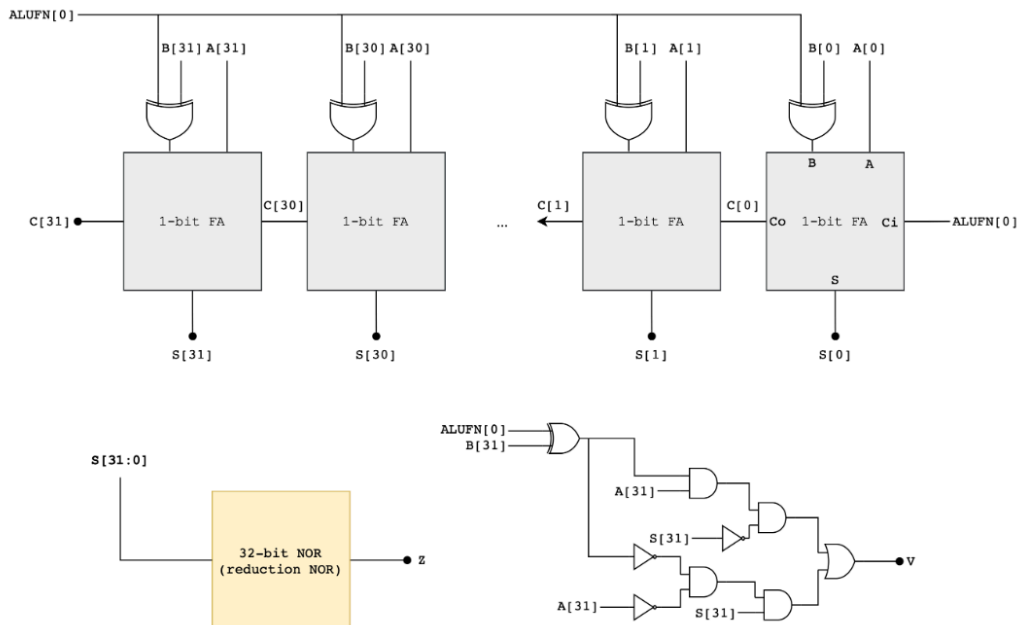
1.1 Adder & Subtractor

```

module adder #(
    SIZE ~ 32 : SIZE > 1
)(
    input a[SIZE],
    input b[SIZE],
    input alufn0,
    output s[SIZE],
    output z,
    output v,
    output n
) {

```

- **Logic:** Inputs a, and (b xor alufn0) into an RCA and outputs s into sum
- ALUFN0 negates b if ALUFN0 is 1
- **Flags (Z, V, N):**
 - Z: NOR(sum) outputs 1 if there's a 1 bit anywhere in the sum, 0 if its empty
 - V: Checks for signed overflow, where either 2 negative numbers added but sum is positive, or 2 positive numbers added but sum is negative
 - N: Checks the MSB if result is negative



Testcases:

```
const TEST_CASES_ADDER = $reverse({  
  // 1. basic add  
  c{32d10,          32d3,          1b0, 32d13,          1b0, 1b0, 1b0},  
  // 2. add to zero  
  c{32d5,          -32d5,          1b0, 32d0,          1b1, 1b0, 1b0},  
  // 3. negative result (no overflow)  
  c{32d3,          -32d10,         1b0, -32d7,          1b0, 1b0, 1b1},  
  // 4. positive overflow (max int + 1)  
  c{32h7FFFFFFF,   32d1,          1b0, 32h80000000,      1b0, 1b1, 1b1},  
  // 5. negative overflow (min int - 1)  
  c{32h80000000,   32d1,          1b1, 32h7FFFFFFF,      1b0, 1b1, 1b0},  
  // 6. subtraction resulting zero  
  c{32d42,         32d42,          1b1, 32d0,          1b1, 1b0, 1b0},  
  // 7. subtraction producing negative  
  c{32d5,          32d8,          1b1, -32d3,          1b0, 1b0, 1b1},  
  // 8. subtraction overflow (large negative minus positive)  
  c{32h80000000,   32d1,          1b0, 32h80000001,      1b0, 1b0, 1b1}  
})
```

1.2 Boolean

- **Operations:** (detail)

```

1  module boolean (
2      input a[32],
3      input b[32],
4      input alufn[6],
5      output bool[32]

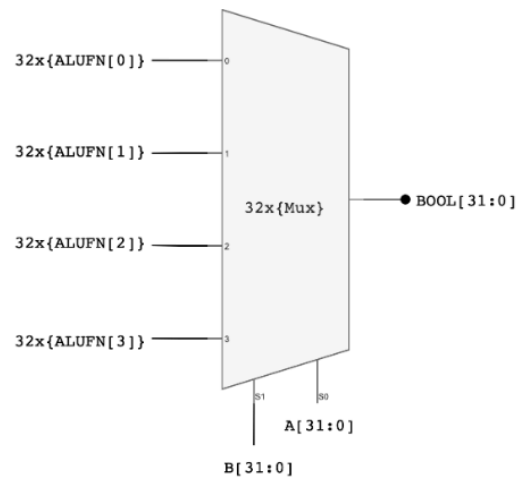
```

- a and b are 32-bit input ports — operands for the boolean operation.
- alufn is a 6-bit control signal —select which boolean operation to perform. Only the lower 4 bits are used here (alufn[3:0])
- bool is a 32-bit output — result of boolean operation

ALUFN[3:0]	Operation
1000	AND
1110	OR
0110	XOR
1010	“A”

Detailed Boolean Unit Schematic

Here's the general schematic of the Boolean Unit:



- **Implementation:** (detail)

```
// declaration of modules utilised in boolean unit
mux_4 mux_4_32[32];
```

Create 32 copies of 4-to-1 multiplexer

```
always{
    // create 32 copies of ALUFN signal as input to each MUX_4 unit
    // the double curly brackets are intentional because
    // we are creating 2D array: 32 by 4 bits
    mux_4_32.in = 32x{{alufn[3:0]}};

    // For each bit, use ALUFN as data input to a 4-to-1 mux
    // Selector = {b[i], a[i]}, Data inputs = {alufn[3:0]}
    repeat(i, 32) {
        // 4-to-1 mux using if/else: selector = {b[i], a[i]}
        mux_4_32.s0[i] = a[i]
        mux_4_32.s1[i] = b[i]
    }

    bool = mux_4_32.out
}
```

Testcases:

```
const TEST_CASES_BOOLEAN = $reverse({
    // 1. AND: basic mask
    c{32hFF00FF00, 32h0F0F0F0F, 4b1000, 32h0F000F00},
    // 2. AND: no common bits
    c{32hAAAAAAAA, 32h55555555, 4b1000, 32h00000000},
    // 3. OR: combine disjoint bits
    c{32hFF00FF00, 32h00FF00FF, 4b1110, 32hFFFFFFFF},
    // 4. OR: one side zero
    c{32h12345678, 32h00000000, 4b1110, 32h12345678},
    // 5. XOR: identical inputs cancel
    c{32hDEADBEEF, 32hDEADBEEF, 4b0110, 32h00000000},
    // 6. XOR: alternating bits
    c{32hAAAAAAAA, 32h55555555, 4b0110, 32hFFFFFFFF},
    // 7. A: pass-through ignores B
    c{32h12345678, 32hFFFFFFFF, 4b1010, 32h12345678},
    // 8. A: zero remains zero
    c{32h00000000, 32hDEADBEEF, 4b1010, 32h00000000}
})
```

1.3 Shifter

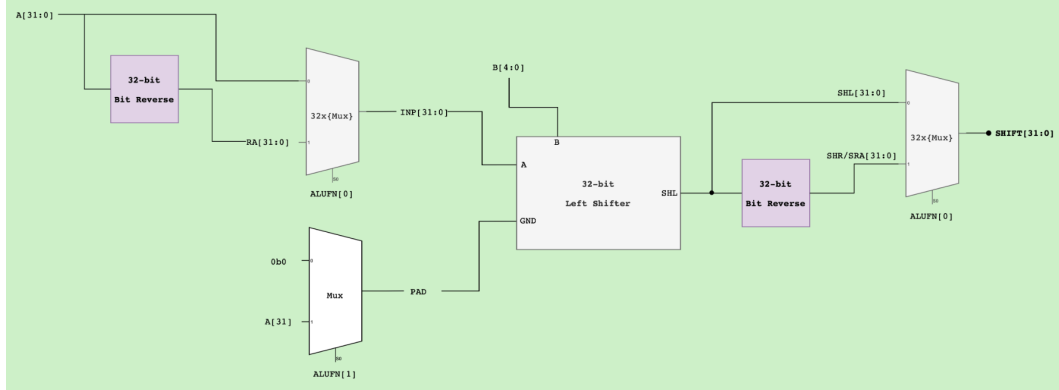
Operation	ALUFN[1:0]
SHL (shift left)	00
SHR (shift right)	01
SRA (shift right with sign extension)	11

- **Types:**
- **Implementation:** Our right shifter was implemented using the left_shifter.luc as a base, where the input bits are reversed when inputted and reversed before outputted as shown in the schematic below

ALTERNATIVE APPROACH: COMPACT SHIFTER

Another approach that **adds** latency but **saves** gates is to use the *left shift logic* for **both** left and right shifts, but for right shifts, **reverse** the bits of the **A** input first on the way in and **reverse** the bits of the output on the way out.

Here's the schematic of this compact shifter.



Testcases:

```
const TEST_CASES_SHIFT = $reverse({
  // 1. SHL: simple left shift
  c{32d1,          32d1,          2b00, 32d2},
  // 2. SHL: overflow bits dropped
  c{32h80000000,   32d1,          2b00, 32h00000000},
  // 3. SHL: shift amount masked (33 -> 1)
  c{32d1,          32d33,         2b00, 32d2},
  // 4. SHR: simple logical right shift
  c{32d8,          32d2,          2b01, 32d2},
  // 5. SHR: negative value zero-filled
  c{32h80000000,   32d1,          2b01, 32h40000000},
  // 6. SHR: shift amount masked (32 -> 0, no shift)
  c{32h80000000,   32d32,         2b01, 32h80000000},
  // 7. SRA: arithmetic right shift with sign extend
  c{32h80000000,   32d1,          2b11, 32hC0000000},
  // 8. SRA: large shift yields all ones
  c{32h80000000,   32d31,         2b11, 32hFFFFFFFF}
})
```


1.4 Comparator

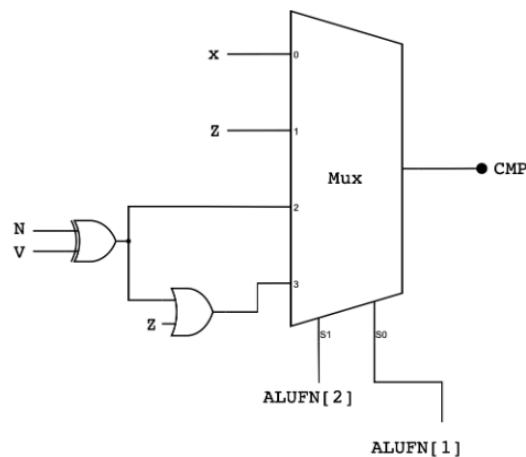
- Logic:

```
compare_unit.z = adder_unit.z
compare_unit.v = adder_unit.v
compare_unit.n = adder_unit.n
compare_unit.alufn = alufn
```

- Takes the input z ,n and v from the adder function (in the ALU module) and outputs z, n or v based on the ALUFN[2:1]

- Dependency:

```
2b01: cmp = z           // CMPEQ: output z
2b10: cmp = n ^ v       // CMPLT: output n XOR v (less than)
2b11: cmp = z | (n ^ v) // CMPLE: output z OR (n XOR v) (less than or equal)
default: cmp = 0
```



Comparison	Equation for LSB	ALUFN2	ALUFN1
$A = B$	$LSB = Z$	0	1
$A < B$	$LSB = N \oplus V$	1	0
$A \leq B$	$LSB = Z + (N \oplus V)$	1	1

- CMPEQ: if A and B are equal, $A + (-B) = 0$, so output z
- CMPLT: Checks if A-B is negative and not overflowed
- CMPLE: CMPEQ or CMPLT

Testcases:

```
const TEST_CASES_COMPARE = $reverse({
  // 1. CMPEQ true (z = 1)
  c{1b1, 1b0, 1b0, 2b01, 1b1},
  // 2. CMPEQ false (z = 0, v/n ignored)
  c{1b0, 1b1, 1b1, 2b01, 1b0},
  // 3. CMPLT true (n ^ v = 1, normal negative)
  c{1b0, 1b0, 1b1, 2b10, 1b1},
  // 4. CMPLT true (n ^ v = 1, overflow sign flip)
  c{1b0, 1b1, 1b0, 2b10, 1b1},
  // 5. CMPLT false (n ^ v = 0)
  c{1b0, 1b0, 1b0, 2b10, 1b0},
  // 6. CMPLE true via equality (z = 1)
  c{1b1, 1b0, 1b0, 2b11, 1b1},
  // 7. CMPLE true via less-than (n ^ v = 1)
  c{1b0, 1b0, 1b1, 2b11, 1b1},
  // 8. CMPLE false (greater-than)
  c{1b0, 1b0, 1b0, 2b11, 1b0}
})
```

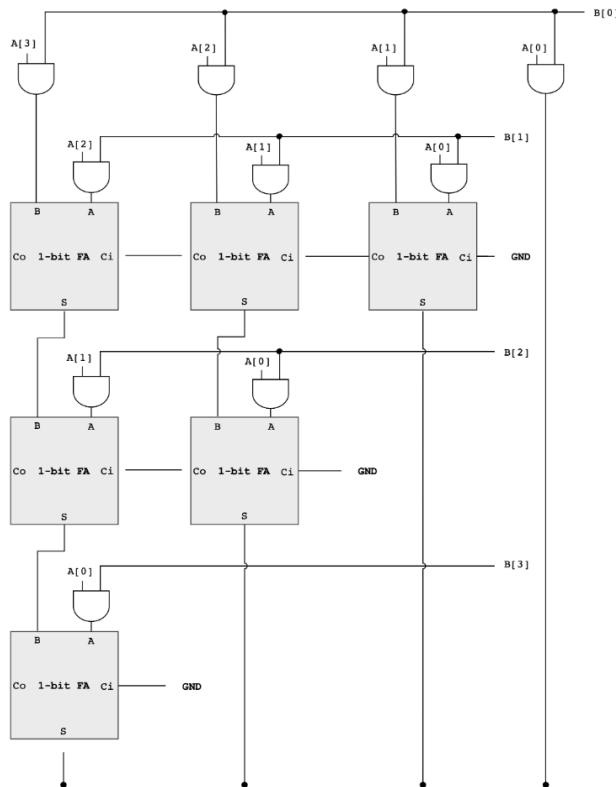
1.5 Multiplier

- 1 Write Down the **Multiplicands**: Write A and B such that each bit of B is **aligned** under each bit of A.
- 2 **Multiply** Each Bit of B by A: Multiply each bit of B with the entire number A, **shifting** the result to the left for each subsequent bit. In binary multiplication, this means we either take A (if the bit in B is 1) or take 0 (if the bit in B is 0).
- 3 Add the **Partial Products**: Add all the partial products together to get the final result.

```

1011 (A = 11 in decimal)
x 1101 (B = 13 in decimal)
-----
1011 (This is A * 1; the rightmost bit in B is 1)
0000 (This is A * 0; shift left by 1 because we're on the second bit from the right)
1011 (This is A * 1; shift left by 2)
1011 (This is A * 1; shift left by 3)
-----
10001111 (Sum of the above partial products)

```



Testcases:

```
const TEST_CASES_MUL = $reverse({
  // 1. small positive multiply
  c{32d3,          32d4,          32d12},
  // 2. multiply by zero
  c{32d12345,      32d0,          32d0},
  // 3. negative times positive
  c{-32d7,         32d6,         -32d42},
  // 4. negative times negative
  c{-32d8,         -32d5,         32d40},
  // 5. large values with truncation (lower 32 bits kept)
  c{32hFFFFFFFF,   32d2,          32hFFFFFFFE},
  // 6. sign bit interaction
  c{32h80000000,   32d2,          32h00000000},
  // 7. power-of-two scaling
  c{32d1024,       32d1024,       32d1048576},
  // 8. overflow wraps around
  c{32h7FFFFFFFF,  32d2,          32hFFFFFFFE}
})
```

1.6 Absolute

```
module absolute (  
    signed input in[32],  
    output abs[32]  
) {  
    always {  
        if (in[31]) {           // If the sign bit is 1 (negative number)  
            // ~in flips all 32 bits simultaneously.  
            // + 1 completes the Two's Complement negation.  
            abs = ~in + 1b1  
        }  
        else {                  // If positive  
            abs = in  
        }  
    }  
}
```

Logic:

- If the MSB of the input is 1, it means the value is negative. To make it positive, force the MSB to be 0, flip the remaining bits and add 1 to the value. This is the 2's complement method.
- If the value is already positive, just output it directly

Testcases:

```

const TESTCASES = $reverse({
  // Zero
  c{32h00000000, 32h00000000},

  // Positive numbers
  c{32h00000005, 32h00000005},
  c{32h00001234, 32h00001234},
  c{32h7FFFFFFF, 32h7FFFFFFF},

  // Negative numbers
  c{32hFFFFFFFF, 32h00000001}, // -1 → 1
  c{32hFFFFFFFB, 32h00000005}, // -5 → 5
  c{32hFFFFFF9C, 32h00000064}, // -100 → 100

  // Larger negative
  c{32hF8A432EB, 32h075BCD15}, // -123456789 → 123456789

  // Edge case
  c{32h80000000, 32h80000000} // depends on implementation
})

```

2. Prototype code + Repo link

https://github.com/Jaydenye-jr/2026_Grp_12_CompStruct_1D

3. Project Management Log: Team Tasks

Code - Jayden, Natalie, Eilees, Salman, Wei Xian

Electronics (Soldering, etc) - Elroy

Electronics Housing - Jarrod

4. Components' Specifications

7 Segments - FJ30101BH ZHIHAO 3" 7 Segment

Unmarked large arcade button

Unmarked 12V buzzer

Generic red, green, yellow and white LEDs